Лабораторна робота №2. Наповнення бази даних і запити мовою SQL

Мета та очікувані результати

- Мета: наповнити схему роботи №1 даними власної предметної області так, щоб повторне наповнення було безпечним; отримувати з бази відповіді на питання замовника запитами з фільтрацією, сортуванням і з'єднанням таблиць; змінювати дані в явних транзакціях і на досліді у двох сеансах показати, від чого залежить, чи збережеться кожна зміна. Дані й запити цієї роботи - основа роботи №3.
- Після виконання роботи студент може: для кожної таблиці назвати обмеження, на якому тримається повторне завантаження, і за потреби додати його міграцією; написати скрипт наповнення, повторний запуск якого не додає рядків і не завершується помилкою, і довести це кількістю рядків до і після; записати на питання замовника запит з умовою, сортуванням і обмеженням кількості рядків і передбачити кількість рядків результату; знайти рядки, які губить умова через NULL, і виправити запит; з'єднати таблиці за парою «зовнішній ключ - первинний ключ», пояснити кількість рядків результату і знайти рядки без пари; змінити й видалити рядки, заздалегідь переглянувши їх запитом з тією самою умовою; показати на даних реакцію на видалення, обрану в роботі №1; виконати операцію з кількох змін у явній транзакції із завершенням COMMIT і ROLLBACK і пояснити, що стається після помилки всередині транзакції; відтворити втрачене оновлення у двох сеансах pgAdmin і показати два способи його уникнути.
- Інструменти: pgAdmin (два вікна Query Tool) із середовищем PostgreSQL роботи №1; Git - здача. Нових інструментів робота не додає.
- Що свідомо не входить у роботу: агрегування і групування, підзапити, віконні функції, індекси й заміри часу запиту, великі обсяги даних - робота №3; досліди з рівнем SERIALIZABLE і взаємним блокуванням не виконуються, їх розбирають контрольні питання до захисту.

Словник термінів роботи

Нових термінів робота не вводить: усі терміни введено на лекціях 1-6. Приклади взято з прокату велосипедів; у звіті відповідні приклади наводяться для власної предметної області.

- **Предметна область** - обмежена частина реального світу, про яку система зберігає дані: її об'єкти, події та правила. Приклад: скрипт наповнення цієї роботи містить рядки тієї самої області, для якої в роботі №1 створено схему. Повторення з лекції 1.
- **Наскрізний проєкт** - формат практикуму: усі п'ять лабораторних робіт виконуються на одній предметній області, і кожна наступна робота розширює результат попередньої. Приклад: таблиці, створені міграцією 001 у роботі №1, у цій роботі отримують рядки, а в роботі №3 над тими самими рядками виконуються аналітичні запити. Повторення з лекції 1.
- **Правило** (бізнес-правило) - твердження вигляду «якщо умова - то дія або висновок», яке визначає, як організація діє в певній ситуації. Приклад: «якщо велосипед повернули на

інший пункт - велосипед закріплюється за пунктом повернення»; це правило виконують дві зміни в одній транзакції. Повторення з лекції 1.

- **Репозиторій** (Git) - каталог проєкту під контролем версій, у якому зберігається історія всіх змін; у курсі - єдиний спосіб здачі робіт. Приклад: окремі коміти зі скриптом наповнення, із запитами і з записами досліду показують, у якому порядку виконано роботу. Повторення з лекції 1.
- **Система керування базами даних** (СУБД) - програмний комплекс, який зберігає дані, надає доступ до них, перевіряє правила, обслуговує одночасну роботу багатьох користувачів і відновлює дані після збою. Приклад: PostgreSQL сама відхиляє видалення клієнта, у якого є видачі, хоча в скрипті роботи немає жодної перевірки. Повторення з лекції 2.
- **Сеанс** (сесія, з'єднання) - установлений на певний час канал зв'язку клієнта із сервером, у межах якого виконуються команди. Приклад: вікна Query Tool «сеанс А» і «сеанс Б» у досліді кроку 8 - два окремі сеанси до бази `dbsa`, і незавершену транзакцію одного з них інший не бачить. Повторення з лекцій 2 і 5.
- **Образ** (Docker image) - готовий незмінний шаблон програми разом з її оточенням; з нього створюються контейнери. Приклад: `postgres:18.6` - той самий образ, що й у роботі №1; для цієї роботи його не змінюють. Повторення з лекції 2.
- **Контейнер** - запущений екземпляр образу: ізольований процес зі власною файловою системою. Приклад: перед початком роботи `docker compose ps` показує контейнер PostgreSQL у стані `running`. Повторення з лекції 2.
- **Том** (volume) - окреме сховище, підключене до контейнера, у якому дані переживають перезапуск і видалення контейнера. Приклад: рядки, додані скриптом наповнення, лишаються на місці після `docker compose down` і повторного `docker compose up -d`. Повторення з лекції 2.
- **Відображення порту** - правило «звертання до порту на моєму комп'ютері передається в контейнер». Приклад: якщо в роботі №1 у файлі опису записано `"5433:5432"`, pgAdmin і в цій роботі підключається до порту 5433. Повторення з лекції 2.
- **Файл опису середовища** (docker compose) - текстовий файл, у якому перелічено, які контейнери запустити, з якими образами, томами й портами; запускається однією командою. Приклад: `compose.yaml` у корені репозиторію, створений у роботі №1, використовується без змін. Повторення з лекції 2.
- **Схема даних** - опис структури бази: які є таблиці, які в них стовпці, якого типу і які правила діють. Приклад: скрипт наповнення схеми не змінює: після нього скрипт перевірки схеми роботи №1 показує ті самі таблиці й обмеження. Повторення з лекції 2.
- **Метадані** (системний каталог) - дані про дані: службові таблиці самої СУБД, у яких зберігається схема. Приклад: скрипт `count-rows.sql` бере перелік таблиць із системного каталогу, тому працює для схеми будь-якої області без змін. Повторення з лекції 2.
- **Таблиця** (відношення) - множина рядків однакової структури, що зберігає дані про однотипні об'єкти. Приклад: таблиця `stations` після наповнення містить чотири пункти. Повторення з лекції 2.
- **Рядок** (запис, кортеж) - один екземпляр об'єкта або події в таблиці. Приклад: видача велосипеда B-12 31 березня о 18:10 - один рядок таблиці `rentals`. Повторення з лекції 2.

- **Стовпець** (поле, атрибут) - одна характеристика, що зберігається для всіх рядків таблиці. Приклад: `returned_at` у таблиці `rentals`. Повторення з лекції 2.
- **Тип даних** - обмеження на множину допустимих значень стовпця. Приклад: `'2024-12-31'` для стовпця `purchase_date` типу `date` означає весь день 31 грудня, а для стовпця `rented_at` типу `timestamp` - лише момент 31 грудня 00:00. Повторення з лекцій 2 і 5.
- **Обмеження** (constraint) - правило, записане в схемі й перевірене самою СУБД. Приклад: команду, яка записує час повернення велосипеда, раніший за час видачі, СУБД відхиляє з повідомленням про обмеження `rentals_check`. Повторення з лекції 2.
- **Транзакція** - група операцій, яку СУБД виконує як одне ціле: або всі, або жодна. Приклад: повернення велосипеда B-12 на пункт №4 - час повернення у видачі і новий пункт у велосипеді. Повторення з лекцій 2 і 5.
- **Атомарність** (A в ACID) - транзакція виконується повністю або не виконується зовсім. Приклад: друга команда повернення велосипеда B-31 посилається на неіснуючий «Пункт №5», і час повернення, записаний першою командою, теж не зберігається. Повторення з лекцій 2 і 5.
- **Ізольованість** (I в ACID) - одночасні транзакції не бачать проміжних результатів одна одної; ступінь цієї ізоляції задається рівнем ізоляції. Приклад: поки сеанс А не виконав COMMIT, сеанс Б бачить стару ціну денного тарифу - 250 грн. Повторення з лекцій 2 і 5.
- **Ідемпотентність** - властивість операції, за якої її повторне виконання не змінює результат. Приклад: після другого запуску скрипта `seed-data.sql` у кожній таблиці стільки ж рядків, скільки після першого. Повторення з лекцій 2 і 5.
- **Первинний ключ** - той потенційний ключ, який обрано основним засобом ідентифікації; він обов'язковий і унікальний. Приклад: повторний рядок «Пункт №2» у команді без ON CONFLICT СУБД відхиляє з повідомленням про обмеження `stations_pkey`. Повторення з лекції 3.
- **Потенційний ключ** (candidate key) - набір атрибутів, що однозначно ідентифікує рядок і не містить надлишкових атрибутів. Приклад: якщо ключ пункту - службовий номер, назва пункту лишається потенційним ключем, і саме на ній тримається повторне завантаження пунктів. Повторення з лекції 3.
- **Природний ключ** і **сурогатний (штучний) ключ** - ключ із реальних даних предметної області і службовий ідентифікатор без змісту, створений системою. Приклад: інвентарний номер велосипеда записують у скрипті наповнення як є, а службовий номер пункту видає СУБД. Повторення з лекції 3.
- **Зовнішній ключ** - атрибут, значення якого має відповідати первинному ключу рядка в іншій таблиці; саме він реалізує зв'язок. Приклад: `rentals.inventory_number` - стовпець, за яким видачу з'єднують із велосипедом. Повторення з лекції 3.
- **Посилальна цілісність** - вимога, щоб зовнішній ключ указував на рядок, який справді існує. Приклад: у скрипті наповнення велосипеди додаються після пунктів, бо кожен велосипед посилається на наявний пункт. Повторення з лекції 3.
- **Реакція на видалення** - правило, що застосовується, коли видаляють рядок, на який є посилання: заборонити, видалити разом (каскадно) або обнулити посилання. Приклад: клієнта К-310, у якого є видачі, видалити не можна, а клієнта К-412 без жодної видачі - можна. Повторення з лекції 3.

- **Значення NULL** - позначка «значення відсутнє або невідоме», а не нуль і не порожній рядок. Приклад: у видачах, які ще тривають, `returned_at` - NULL; у клієнтів К-310 і К-412 не вказано телефону. Повторення з лекцій 3 і 6.
- **Складений ключ** - первинний ключ із двох і більше атрибутів. Приклад: `ON CONFLICT (inventory_number, rented_at)` у команді наповнення видач перелічує обидва стовпці ключа. Повторення з лекції 4.
- **Нормалізація** - послідовне розкладання таблиць на менші так, щоб усунути аномалії, зберігши всі дані та зв'язки між ними. Приклад: відповідь «адреса пункту для кожної видачі» потребує з'єднання, бо після нормалізації адреса зберігається лише в таблиці `stations`. Повторення з лекції 4.
- **Декомпозиція без втрат** - розділення, після якого з'єднання отриманих таблиць повертає точно вихідні дані, без зайвих і без загублених рядків. Приклад: з'єднання видач із клієнтами, велосипедами, пунктами і тарифами за ключами повертає 10 рядків - по одному на кожну видачу. Повторення з лекції 4.
- **SQL** (мова структурованих запитів) - мова, якою описують структуру реляційної бази, змінюють дані та отримують їх. Приклад: скрипт наповнення, запити, зміни даних і досліди з транзакціями - файли з командами SQL. Повторення з лекції 5.
- **Команди зміни даних** (DML) - частина SQL, яка додає, змінює і видаляє рядки: INSERT, UPDATE, DELETE; до цієї самої групи належить вибірка SELECT. Приклад: у файлі `data-changes.sql` лише команди UPDATE, DELETE і SELECT та команди керування транзакцією, без жодної CREATE чи ALTER. Повторення з лекцій 5 і 6.
- **CREATE TABLE** (створити таблицю) - команда, яка додає до схеми нову таблицю зі стовпцями, типами й обмеженнями. Приклад: команди INSERT у скрипті наповнення йдуть у тому самому порядку, що й команди CREATE TABLE у файлі `001_create_schema.up.sql`. Повторення з лекції 5.
- **DROP TABLE** (видалити таблицю) - команда, яка видаляє таблицю разом з усіма її даними. Приклад: цикл down - up очищає базу від рядків, бо файл down видаляє таблиці командами DROP TABLE, а файл up створює їх знову порожніми. Повторення з лекції 5.
- **ALTER TABLE** (змінити таблицю) - команда, яка змінює наявну таблицю, не втрачаючи її даних: додає чи видаляє стовпець або обмеження. Приклад: `ALTER TABLE stations ADD CONSTRAINT stations_station_name_key UNIQUE (station_name);`. Повторення з лекції 5.
- **PRIMARY KEY**, **NOT NULL**, **UNIQUE**, **CHECK** - обмеження в команді: первинний ключ; обов'язкове значення; значення не повторюється; умова, яку задовольняє кожен рядок. Приклад: на UNIQUE номера рами тримається повторне завантаження велосипедів так само, як на первинному ключі - інвентарному номері. Повторення з лекції 5.
- **REFERENCES** і **ON DELETE** - обмеження, яке реалізує зовнішній ключ, і його частина, що задає реакцію на видалення: RESTRICT - заборонити, CASCADE - видалити разом, SET NULL - обнулити посилання. Приклад: `ON DELETE RESTRICT` у стовпці `station_name` таблиці `bikes` не дає видалити «Пункт №7», за яким закріплено велосипеди. Повторення з лекції 5.
- **Стовпець ідентичності** (identity column) - стовпець, значення якого СУБД видає сама за лічильником; так реалізують сурогатний ключ. Приклад: номер СУБД витрачає і на рядок, пропущений через ON CONFLICT; скрипт із двома пунктами запустили двічі, потім дописали третій пункт - у робочій базі він отримує номер 7, а в порожній базі - 3. Повторення з лекції 5.

- **Файл міграції** (migration) - файл із командами однієї зміни схеми; файли виконуються по черзі за номером, а вже виконаний файл не редагують. Приклад: унікальне обмеження, якого бракує для повторного завантаження, додає міграція `002_add_station_name_unique`, а не правка файлу 001. Повторення з лекції 5.
- **Зворотна міграція** (down migration) - файл, який точно скасовує зміну схеми, виконану парним файлом up, і повертає схему до стану перед ним. Приклад: `002_add_station_name_unique.down.sql` видаляє обмеження командою `ALTER TABLE stations DROP CONSTRAINT stations_station_name_key;`. Повторення з лекції 5.
- **INSERT** (додати) - команда, яка додає рядки до таблиці. Приклад: одна команда INSERT у скрипті наповнення додає всі шість велосипедів переліком рядків після VALUES. Повторення з лекції 5.
- **UPDATE** (змінити) - команда, яка змінює значення в наявних рядках. Приклад: `UPDATE bikes SET condition_rating = 5 WHERE model = 'City' AND condition_rating <= 4;` змінює два рядки. Повторення з лекції 5.
- **DELETE** (видалити) - команда, яка видаляє рядки з таблиці. Приклад: `DELETE FROM customers WHERE registration_date >= '2026-03-01';` видаляє одного клієнта - К-412. Повторення з лекції 5.
- **ON CONFLICT DO NOTHING** (у разі конфлікту нічого не робити) - доповнення до INSERT: рядок, який порушив би первинний ключ чи обмеження UNIQUE, мовчки пропускається, а решта рядків додається. Приклад: повторний запуск команди наповнення пунктів з `ON CONFLICT (station_name) DO NOTHING` повідомляє `INSERT 0 0`. Повторення з лекції 5.
- **UPSERT** (INSERT ... ON CONFLICT DO UPDATE, «вставити або оновити») - доповнення до INSERT: якщо рядок із таким ключем уже є, його оновлюють значеннями нового рядка, доступними під іменем EXCLUDED. Приклад: з `ON CONFLICT (tariff_code) DO UPDATE SET price = EXCLUDED.price` повторний запуск скрипта повернув би ціні тарифу значення зі скрипта, а з DO NOTHING лишає ціну, яка вже є в базі. Повторення з лекції 5.
- **BEGIN** (почати) - команда, яка відкриває явну транзакцію: наступні команди сеансу виконуються як одне ціле до COMMIT або ROLLBACK. Приклад: BEGIN перед зміною оцінки стану велосипедів дає змогу перевірити результат запитом SELECT і лише потім обрати COMMIT чи ROLLBACK. Повторення з лекції 5.
- **COMMIT** (підтвердити) - команда, яка завершує транзакцію і робить усі її зміни постійними й видимими іншим сеансам. Приклад: після COMMIT сеансу А сеанс Б, що чекав на блокування, завершує свою команду UPDATE. Повторення з лекції 5.
- **ROLLBACK** (скасувати) - команда, яка завершує транзакцію і скасовує всі її зміни. Якщо в транзакції сталася помилка, PostgreSQL не виконує наступних команд, а COMMIT такої транзакції теж завершується скасуванням. Приклад: час повернення велосипеда В-31, записаний у транзакції, після ROLLBACK знову NULL. Повторення з лекції 5.
- **Режим автопідтвердження** (autocommit) - режим, у якому кожна команда поза явною транзакцією виконується як окрема транзакція і підтверджується одразу. Приклад: у pgAdmin 9.17 за замовчуванням позначено пункт Auto commit?, тому команда DELETE, виконана без BEGIN, підтверджується одразу. Повторення з лекції 5.
- **Втрачене оновлення** (lost update) - ситуація, коли два сеанси прочитали те саме значення, кожен обчислив на його основі нове, і зміну того, хто записав першим, затер другий.

Приклад: менеджер А знижує ціну денного тарифу на 10 %, менеджер Б підвищує її на 30 грн; обидва прочитали 250, і в базі лишається 280 замість 255. Повторення з лекції 5.

- **Блокування** (lock) - позначка, якою СУБД закриває рядок, змінений незавершеною транзакцією: інша транзакція, що хоче змінити той самий рядок, чекає на її COMMIT або ROLLBACK. Приклад: UPDATE ціни тарифу Д1 у вікні сеансу Б не завершується, доки у вікні сеансу А не виконано COMMIT. Повторення з лекції 5.
- **Рівень ізоляції** (isolation level) - налаштування транзакції, яке визначає, які зміни одночасних транзакцій вона бачить і як СУБД реагує на конфлікт. Приклад: `BEGIN ISOLATION LEVEL REPEATABLE READ;` на початку обох транзакцій другого досліду кроку 8. Повторення з лекції 5.
- **READ COMMITTED** (читання підтвердженого) - рівень ізоляції PostgreSQL за замовчуванням: кожна команда бачить дані, підтверджені на момент її початку. Приклад: у першому досліді кроку 8 сеанс Б після COMMIT сеансу А записує 280 без жодного повідомлення. Повторення з лекції 5.
- **REPEATABLE READ** (повторюване читання) - рівень ізоляції, за якого вся транзакція бачить дані такими, якими вони були на момент її першої команди; якщо транзакція змінює рядок, який після цього змінила й підтвердила інша транзакція, PostgreSQL завершує команду помилкою серіалізації. Приклад: у другому досліді сеанс Б замість запису 280 отримує помилку. Повторення з лекції 5.
- **SERIALIZABLE** (серіалізований) - найсуворіший рівень ізоляції: результат одночасних транзакцій має бути таким, ніби вони виконувалися по черзі. Приклад: правило «у клієнта не більше однієї незавершеної видачі»; два пункти одночасно перевіряють, що незавершених видач у клієнта К-114 немає, і кожен додає свою; на REPEATABLE READ обидві транзакції підтверджуються, а на SERIALIZABLE COMMIT другої завершується помилкою серіалізації. Повторення з лекції 5.
- **Помилка серіалізації** (serialization failure) - відмова PostgreSQL завершити транзакцію, бо паралельна транзакція вже змінила дані, на які вона спиралася; таку транзакцію скасовують і виконують повторно з початку. Приклад: після повідомлення `could not serialize access due to concurrent update` сеанс Б виконує ROLLBACK, повторює транзакцію, читає вже 225 і записує 255. Повторення з лекції 5.
- **Взаємне блокування** (deadlock) - ситуація, коли дві транзакції чекають одна на одну: кожна тримає рядок, потрібний іншій; PostgreSQL перериває одну з них з помилкою `deadlock detected`. Приклад: сеанс А змінює велосипед В-12, потім В-44, а сеанс Б - спершу В-44, потім В-12. Повторення з лекції 5.
- **Запит** (query) - команда SELECT, яка повертає дані з бази у вигляді таблиці-результату і нічого в базі не змінює. Приклад: запит «які пункти не мають жодного велосипеда» повертає «Пункт №9», і жодна таблиця від нього не змінюється. Повторення з лекції 6.
- **SELECT** (вибрати) - команда вибірки даних; частини записуються в порядку: список вибірки, FROM, WHERE, ORDER BY, LIMIT. Приклад: `SELECT inventory_number, model, purchase_date FROM bikes WHERE purchase_date BETWEEN '2024-01-01' AND '2024-12-31';`. Повторення з лекції 6.
- **FROM** (з) - частина SELECT, яка вказує таблицю або з'єднані таблиці, з яких беруться рядки. Приклад: `FROM stations AS s LEFT JOIN bikes AS b ON b.station_name = s.station_name` - рядки беруться з двох з'єднаних таблиць. Повторення з лекції 6.

- **WHERE** (умова) - частина SELECT, UPDATE чи DELETE, яка лишає тільки ті рядки, для яких умова істинна. Приклад: умова `model = 'City' AND condition_rating <= 4` спершу в SELECT показує два велосипеди, а потім відбирає ті самі два рядки в UPDATE. Повторення з лекції 6.
- **Предикат** (predicate) - умова, яку перевіряють для кожного рядка окремо і яка дає істину, хибність або невідомо. Приклад: `purchase_date >= '2024-01-01'` перевіряється для кожного велосипеда. Повторення з лекції 6.
- **AND, OR, NOT** (логічні оператори) - оператори, які поєднують предикати: AND - мають виконуватися обидві умови, OR - хоча б одна, NOT - заперечення умови; AND виконується раніше за OR. Приклад: `phone NOT LIKE '050%' OR phone IS NULL` - клієнти, чий номер не починається з 050, і клієнти без телефону. Повторення з лекції 6.
- **BETWEEN** (між) - предикат «значення лежить у межах від ... до ...», обидві межі включно. Приклад: для стовпця `purchase_date` типу `date` умова `BETWEEN '2024-01-01' AND '2024-12-31'` охоплює весь 2024 рік, а для `rented_at` типу `timestamp` той самий запис загубив би видачі 31 грудня після 00:00. Повторення з лекції 6.
- **IN** (у переліку) - предикат «значення збігається з одним зі значень списку». Приклад: `tariff_code IN ('Д1', 'Т1')` - видачі за денним або тижневим тарифом. Повторення з лекції 6.
- **LIKE** (схоже на) - предикат порівняння тексту із шаблоном: `%` - будь-яка кількість будь-яких символів, `_` - рівно один символ; у PostgreSQL розрізняє великі й малі літери. Приклад: `phone LIKE '050%'` - клієнти з номером, що починається з 050. Повторення з лекції 6.
- **ORDER BY** (упорядкувати за) - частина SELECT, яка задає порядок рядків результату: ASC - за зростанням, DESC - за спаданням. Приклад: `ORDER BY purchase_date DESC` - спершу найновіші велосипеди. Повторення з лекції 6.
- **LIMIT** (обмежити) - частина SELECT, яка лишає не більше вказаної кількості рядків результату; однозначний результат дає лише разом з ORDER BY, який не залишає нічиїх. Приклад: `ORDER BY purchase_date DESC LIMIT 2` - два найновіші велосипеди, B-44 і B-40. Повторення з лекції 6.
- **NULLS FIRST, NULLS LAST** (NULL на початку, NULL у кінці) - доповнення до ORDER BY, яке ставить рядки з NULL на початок або в кінець результату; у PostgreSQL за спаданням такі рядки йдуть першими. Приклад: `ORDER BY returned_at NULLS FIRST` - спершу видачі, які ще тривають. Повторення з лекції 6.
- **IS NULL, IS NOT NULL** (є NULL, не є NULL) - предикати перевірки на відсутність значення; єдиний спосіб знайти рядки з NULL. Приклад: `WHERE b.inventory_number IS NULL` після лівого з'єднання пунктів і велосипедів лишає пункт без жодного велосипеда. Повторення з лекції 6.
- **Трозначна логіка** (three-valued logic) - правило SQL, за яким умова дає істину, хибність або невідомо; порівняння з NULL дає «невідомо», а WHERE лишає лише рядки з істиною. Приклад: для клієнтів без телефону умова `phone NOT LIKE '050%'` дає «невідомо», тому запит повертає лише двох клієнтів, а не чотирьох. Повторення з лекції 6.
- **З'єднання таблиць** (JOIN) - частина запиту, яка ставить поруч рядки двох таблиць, для яких виконується умова з'єднання, записана після ON; зазвичай умова - рівність зовнішнього ключа однієї таблиці і первинного ключа іншої. Приклад: `rentals AS r JOIN`

`bikes AS b ON b.inventory_number = r.inventory_number` ставить модель велосипеда поруч із кожною видачею. Повторення з лекції 6.

- **Внутрішнє з'єднання** (INNER JOIN, або просто JOIN) - з'єднання, результат якого містить лише рядки, що знайшли пару в обох таблицях. Приклад: з'єднання 10 видач із велосипедами повертає 10 рядків, і велосипеда B-19, якого жодного разу не видавали, у результаті немає. Повторення з лекції 6.
- **Ліве зовнішнє з'єднання** (LEFT JOIN) - з'єднання, яке лишає всі рядки лівої таблиці; якщо пари немає, стовпці правої таблиці заповнюються NULL. Приклад: `stations AS s LEFT JOIN bikes AS b` повертає 7 рядків: 6 велосипедів разом із пунктами і «Пункт №9» з NULL замість інвентарного номера. Повторення з лекції 6.
- **Псевдонім** (alias) - коротке ім'я таблиці чи стовпця, чинне в межах одного запиту; задається словом AS і пишеться англійською. Приклад: у запиті `FROM rentals AS r JOIN bikes AS b` стовпець `r.station_name` - пункт видачі, а `b.station_name` - пункт, за яким велосипед закріплено зараз. Повторення з лекції 6.
- **Логічний порядок виконання запиту** - порядок, у якому СУБД опрацьовує частини SELECT: FROM і JOIN, WHERE, SELECT, ORDER BY, LIMIT. Приклад: запит про пункти без велосипедів читається так: FROM і LEFT JOIN - пункти поруч із їхніми велосипедами, WHERE - лише рядки без пари, SELECT - назва й адреса пункту. Повторення з лекції 6.

Зв'язок з лекціями

Роботу виконують після лекції 6 на схемі, створеній у роботі №1. Завдання для самостійної роботи - чернетки кроків цієї роботи: їх доопрацьовують за вимогами нижче, а не створюють заново.

Лекція	Що потрібно для роботи	Чернетка із завдання для самостійної роботи
2	транзакція, атомарність, ізольованість, ідемпотентність, сеанс	-
3	первинний і зовнішній ключі, NULL, посилальна цілісність, реакція на видалення	реакції на видалення - перевіряються на даних на кроці 6
4	нормалізація, декомпозиція без втрат	«погана» таблиця і кроки нормалізації - перелік таблиць для з'єднання на кроці 5
5	INSERT, UPDATE, DELETE, ON CONFLICT, ALTER TABLE і міграції, BEGIN, COMMIT, ROLLBACK, втрачене оновлення, блокування, рівні ізоляції	скрипт наповнення двох таблиць з ON CONFLICT DO NOTHING і записи двох запусків (завдання 4); втрачене оновлення у двох вікнах pgAdmin і повтор на REPEATABLE READ (завдання 5)
6	SELECT, умова WHERE і предикати, ORDER BY і LIMIT, NULL у запитах, з'єднання таблиць, псевдоніми, логічний порядок виконання	шість запитів на питання замовника (завдання 1); запит, що губить рядки з NULL (завдання 2); три з'єднання (завдання 3)

Підготовка середовища

Робота виконується в середовищі роботи №1: нових програм встановлювати не потрібно.

1. Запустити Docker Desktop (Windows, macOS) і дочекатися стану «Engine running»; у Linux має працювати служба Docker.
2. У терміналі (у Windows - PowerShell або Windows Terminal) перейти в корінь репозиторію і виконати:

   ```
   docker compose up -d
   docker compose ps
   ```

   Контейнер має бути в стані `running`.
3. У pgAdmin підключитися до сервера, зареєстрованого в роботі №1, і відкрити Query Tool для бази `dbsa`. Якщо в роботі №1 порт змінено на 5433, підключення йде до того самого порту.
4. Переконатися, що схема на місці: виконати скрипт перевірки схеми (`lab-01/check-schema.sql`) роботи №1. Результат має збігатися з розділом 10 звіту роботи №1. Якщо таблиць немає, наприклад після `docker compose down -v`, схему відтворюють: виконують файли `*.up.sql` з каталогу `lab-01/migrations` за зростанням номерів.
5. Переконатися, що таблиці порожні: виконати скрипт підрахунку рядків (`lab-02/count-rows.sql`) - у кожній таблиці 0 рядків. Якщо рядки лишилися від попередніх спроб, базу очищають циклом down - up: виконують файли `*.down.sql` за спаданням номерів, потім `*.up.sql` за зростанням. Файли down видаляють таблиці разом із даними.
6. Перевірити параметри Query Tool, від яких залежать кроки 7 і 8: у меню кнопки Execute options (стрілка поруч із кнопкою виконання) пункт Auto commit? має бути позначено, а пункт Auto rollback on error? - ні. Саме такі значення pgAdmin 9.17 має за замовчуванням. Якщо Auto rollback on error? позначено, після помилки pgAdmin сам скасовує транзакцію, і на кроці 7 не видно, як поводиться PostgreSQL.

Для кроку 8 потрібне друге вікно Query Tool до тієї самої бази: у дереві об'єктів виділити базу `dbsa` і ще раз відкрити Query Tool (меню Tools - Query Tool). Кожне вікно - окремий сеанс.

Якщо середовище не запускається, діють поради розділу «Типові помилки та як їх уникнути» роботи №1 (`lab-01.pdf`); до викладача звертаються до початку роботи, як і в роботі №1.

Порядок виконання

Крок 1. Ключі для повторного завантаження

- Створити в репозиторії каталог `lab-02`.
- Для кожної таблиці схеми визначити обмеження, яке не пустить повторний рядок під час другого запуску скрипта наповнення: первинний ключ із реальних даних або UNIQUE на

потенційному ключі. Перелік обмежень кожної таблиці показує скрипт перевірки схеми (рядки PRIMARY KEY і UNIQUE). Стовпці цього обмеження стануть переліком у `ON CONFLICT (...)` на кроці 2.

- Первинний ключ - стовпець ідентичності від повтору не захищає: СУБД видає кожному рядку новий номер, тому повторний рядок отримує інший ключ і конфлікту не виникає. Для такої таблиці потрібне UNIQUE на потенційному ключі з реальних даних: назві, номері документа, парі «клієнт - час події».
- Якщо потрібного обмеження в схемі немає, його додають міграцією в каталозі `lab-02/migrations` з наступним вільним номером (після роботи №1 це зазвичай 002). Міграцію виконують до наповнення: у таблиці, де вже є однакові значення, СУБД обмеження не створить. Назву обмеження записують явно, щоб файл down міг видалити його за назвою. Зразок нижче - для варіанта схеми, у якому ключ пункту - службовий номер `station_id`, а не назва, як у зразку роботи №1.

```
-- 002_add_station_name_unique.up.sql
-- Назва пункту - потенційний ключ: на ньому тримається повторне завантаження пунктів,
-- бо первинний ключ station_id - стовпець ідентичності.

ALTER TABLE stations
    ADD CONSTRAINT stations_station_name_key UNIQUE (station_name);
```

```
-- 002_add_station_name_unique.down.sql
-- Скасування 002_add_station_name_unique.up.sql.

ALTER TABLE stations
    DROP CONSTRAINT stations_station_name_key;
```

- Міграцію перевіряють циклом up - down - up у базі `dbsa`, поки таблиці порожні: після up скрипт перевірки схеми показує рядок `UNIQUE | stations_station_name_key | UNIQUE (station_name)`, після down цього рядка немає, після повторного up він знову є.

Результат: розділ 1 звіту - таблиця «таблиця - обмеження - стовпці ON CONFLICT»; за потреби - два файли міграції.

Крок 2. Скрипт наповнення

- Створити файл `lab-02/seed-data.sql`. Файл починається коментарем, що пояснює, які дані він додає.
- Для кожної таблиці - одна команда INSERT з переліком стовпців і переліком рядків після VALUES і доповненням `ON CONFLICT (стовпці з кроку 1) DO NOTHING`. Команди йдуть у тому самому порядку, що й CREATE TABLE у файлі up: спершу таблиці, на які посилаються. Скрипт лише додає рядки: команд UPDATE, DELETE, CREATE, ALTER чи DROP у ньому немає.

```sql
-- seed-data.sql
-- Наповнення бази прокату: пункти, клієнти, тарифи, велосипеди, видачі (фрагмент для
зразка оформлення).
-- Повторний запуск не додає рядків і не завершується помилкою.

INSERT INTO stations (station_name, address, city)
VALUES ('Пункт №2', 'Паркова, 5', 'Харків'),
       ('Пункт №4', 'Садова, 1', 'Харків'),
       ('Пункт №7', 'Липова, 3', 'Харків'),
       ('Пункт №9', 'Лісна, 12', 'Харків')      -- пункт без велосипедів
ON CONFLICT (station_name) DO NOTHING;

INSERT INTO bikes (inventory_number, frame_number, model, purchase_date, condition_rating,
station_name)
VALUES ('B-12', 'FR-1012', 'City', '2022-05-18', 4, 'Пункт №2'),
       ('B-19', 'FR-1019', 'City', '2024-03-02', 5, 'Пункт №7'),
       ('B-27', 'FR-1027', 'City', '2023-04-12', 3, 'Пункт №4'),
       ('B-31', 'FR-1031', 'Trek', '2023-05-03', 5, 'Пункт №4'),
       ('B-40', 'FR-1040', 'Trek', '2024-06-21', 4, 'Пункт №7'),
       ('B-44', 'FR-1044', 'Cube', '2024-09-10', 2, 'Пункт №2')
ON CONFLICT (inventory_number) DO NOTHING;
```

- Вимоги до даних:
  - у кожній таблиці щонайменше 3 рядки, у таблиці з найбільшою кількістю зовнішніх ключів (зазвичай це таблиця подій: видачі, замовлення, записи) - щонайменше 10;
  - щонайменше в одному стовпці без NOT NULL є і рядки з NULL, і рядки зі значенням;
  - щонайменше на один рядок не посилається жоден рядок іншої таблиці (пункт без велосипедів, клієнт без видач) - він потрібен для лівого з'єднання;
  - якщо в схемі є стовпець типу `timestamp`, рядки охоплюють щонайменше два місяці, і хоча б один рядок припадає на останній день місяця пізніше за 00:00;
  - дані вигадані: справжніх прізвищ, номерів телефонів і адрес знайомих людей не використовують.
- Якщо таблиця посилається на таблицю із сурогатним ключем, у дочірніх рядках записують номери, які СУБД видає в порожній базі під час першого запуску скрипта: 1, 2, 3 - у порядку рядків після VALUES. СУБД витрачає номери й на рядки, пропущені через ON CONFLICT, тому після кожної зміни скрипта базу `dbsa` очищають циклом down - up міграцій і наповнюють знову; збіг перевіряє крок 9.
- Виконати скрипт у Query Tool для бази `dbsa` цілком. Якщо виконання зупинилося на помилці, скрипт виправляють і запускають знову: рядки, які вже додано, повторно не додаються.
- Виконати скрипт підрахунку рядків і записати результат.

Результат: файл `seed-data.sql` і розділ 2 звіту - кількість рядків у кожній таблиці і перелік рядків, доданих для вимог до даних.

Крок 3. Перевірка ідемпотентності

- Кількість рядків після першого запуску вже записано на кроці 2.
- Запустити `seed-data.sql` цілком удруге: виконання має завершитися без помилки. Знову виконати скрипт підрахунку рядків: результат має збігатися з першим.
- Виділити в скрипті команду INSERT для таблиці подій і виконати лише її: на вкладці Messages - `INSERT 0 0`, тобто нуль нових рядків.
- Скопіювати ту саму команду в Query Tool без рядка `ON CONFLICT ...` (у файл скрипта цю зміну не записувати) і виконати: СУБД відхиляє команду повідомленням `duplicate key value violates unique constraint "..."`. Назва обмеження в повідомленні - те обмеження, яке на кроці 1 обрано для цієї таблиці.

```
-- Та сама команда без ON CONFLICT.
-- Очікувано: duplicate key value violates unique constraint "stations_pkey"
INSERT INTO stations (station_name, address, city)
VALUES ('Пункт №2', 'Паркова, 5', 'Харків'),
       ('Пункт №9', 'Лісна, 12', 'Харків');
```

Результат: розділ 3 звіту - таблиця «дія - повідомлення СУБД - кількість рядків у таблицях» і речення про те, яке обмеження захистило кожну таблицю від дублікатів.

Крок 4. Запити з фільтрацією і сортуванням

- Створити файл `lab-02/queries.sql`. Сформулювати словами шість питань замовника власної області і записати на кожне запит SELECT.
- Разом шість запитів використовують: порівняння з AND або OR, BETWEEN, IN, LIKE, IS NULL або IS NOT NULL, ORDER BY з LIMIT. Щонайменше один запит поєднує AND і OR - з дужками.
- Перед кожним запитом - коментар із номером, питанням словами і кількістю рядків результату. Кількість рядків pgAdmin показує в рядку стану під результатом (`Total rows`).
- Правила запису:
  - кожна умова і лишає, і відкидає хоча б один рядок; якщо ні, дані на кроці 2 доповнюють;
  - період для стовпця з датою й часом записують напівінтервалом: `>=` початок періоду і `<` початок наступного;
  - текст і дати - в одинарних лапках;
  - LIMIT пишеться лише разом з ORDER BY; якщо стовпець сортування може повторюватися, до сортування додають стовпці ключа.

```sql
-- Запит 3. Які видачі за денним тарифом відбулися в березні 2026 року?
-- Рядків: 3
SELECT inventory_number, rented_at, customer_number
FROM rentals
WHERE tariff_code = 'Д1'
  AND rented_at >= '2026-03-01' AND rented_at < '2026-04-01'
ORDER BY rented_at, inventory_number;
```

- Кожен запит виконують окремо: виділити його разом із коментарем і виконати.
- Окремо записати пару запитів для стовпця без NOT NULL: запит з порівнянням, який губить рядки з NULL, і виправлений запит. У звіті пояснити різницю трозначною логікою.

```sql
-- Клієнти, чий телефон не починається з 050.
-- Рядків: 2 - клієнтів без телефону немає: для NULL умова дає «невідомо».
SELECT customer_number, last_name, phone
FROM customers
WHERE phone NOT LIKE '050%';

-- Виправлено. Рядків: 4
SELECT customer_number, last_name, phone
FROM customers
WHERE phone NOT LIKE '050%' OR phone IS NULL;
```

Результат: запити у файлі `queries.sql` і розділ 4 звіту - таблиця «питання - використані конструкції - кількість рядків» і пояснення пари запитів із NULL.

Крок 5. Запити зі з'єднанням таблиць

- Дописати у файл `queries.sql` три запити:
  1. внутрішнє з'єднання двох таблиць за парою «зовнішній ключ - первинний ключ»;
  2. з'єднання всіх таблиць, на які під час нормалізації в роботі №1 (розділ 5 звіту) розкладено «погану» таблицю: результат містить ті самі стовпці, що й «погана» таблиця;
  3. ліве зовнішнє з'єднання, яке знаходить рядки без пари, - умова IS NULL на ключі правої таблиці.
- У запиті зі з'єднанням кожен стовпець записують із псевдонімом таблиці. Умову ON беруть зі схеми: зовнішній ключ однієї таблиці дорівнює первинному ключу іншої; для складеного ключа в умові всі його стовпці.
- Для кожного запиту в коментарі й у звіті - кількість рядків і пояснення, чому вона саме така: які правила схеми (NOT NULL зовнішнього ключа, REFERENCES, первинний ключ) гарантують, що кожен рядок знаходить рівно одну пару. Якщо зовнішній ключ допускає NULL, внутрішнє з'єднання такі рядки втрачає - це теж пояснюється.
- Перевірка умови з'єднання: з'єднання за зовнішнім ключем не може дати більше рядків, ніж є в таблиці із зовнішнім ключем після фільтра. Більше рядків означає помилку в ON.

```sql
-- З’єднання 3. Які пункти не мають жодного закріпленого велосипеда?
-- Рядків: 1. Без WHERE ліве з’єднання дає 7 рядків: 6 велосипедів, кожен закріплений
-- рівно за одним пунктом, і «Пункт №9» без пари; WHERE лишає лише рядок без пари.
SELECT s.station_name, s.address
FROM stations AS s
LEFT JOIN bikes AS b ON b.station_name = s.station_name
WHERE b.inventory_number IS NULL;
```

- Для з’єднання 2 записати в звіті, як запит читається в логічному порядку виконання: FROM і JOIN, WHERE, SELECT, ORDER BY, LIMIT - що відбувається з рядками на кожному кроці. Порівняти результат із «поганою» таблицею роботи №1: чи збігаються стовпці і чи по одному рядку результату на кожну подію.

Результат: розділ 5 звіту - три запити з кількістю рядків і поясненням, прочитання з’єднання 2 в логічному порядку і висновок порівняння з «поганою» таблицею.

Крок 6. Зміна і видалення даних

- Кількість рядків запитів кроків 4 і 5 записано до цього кроку: зміни даних її змінюють, а в порожній базі її перевіряє крок 9.
- Створити файл `lab-02/data-changes.sql` із двома змінами UPDATE і одним видаленням DELETE, які мають сенс для області (огляд, закриття, виправлення помилки введення). Щонайменше одна умова - не рівність ключа, а порівняння, IN, BETWEEN чи LIKE.
- Кожну зміну виконують командами по одній за шаблоном: BEGIN; SELECT з тією самою умовою WHERE - рядки, які буде змінено; UPDATE або DELETE - повідомлення `UPDATE N` чи `DELETE N`, де N збігається з кількістю рядків SELECT; SELECT - результат зміни; COMMIT, якщо результат очікуваний, інакше ROLLBACK.

```sql
-- Зміна 1. Після планового огляду велосипедам City з оцінкою стану 4 і нижче виставлено
5.
BEGIN;
SELECT inventory_number, model, condition_rating
FROM bikes
WHERE model = 'City' AND condition_rating <= 4;         -- 2 рядки: B-12, B-27
UPDATE bikes SET condition_rating = 5
WHERE model = 'City' AND condition_rating <= 4;         -- UPDATE 2
SELECT inventory_number, model, condition_rating
FROM bikes
WHERE model = 'City';                                   -- у всіх трьох велосипедів City -
5
COMMIT;
```

- Реакції на видалення: для кожної реакції, обраної в роботі №1 (RESTRICT, CASCADE, SET NULL), дописати у файл одне видалення рядка, на який посилаються інші рядки, у транзакції із завершенням ROLLBACK:

- RESTRICT - DELETE відхиляється повідомленням `violates RESTRICT setting of foreign key constraint`; після помилки транзакція не виконує команд, тому її завершують ROLLBACK;
- CASCADE - після DELETE запит до залежної таблиці не знаходить пов'язаних рядків; ROLLBACK повертає всі рядки;
- SET NULL - після DELETE у пов'язаних рядках зовнішній ключ - NULL; ROLLBACK повертає значення.

```
-- Реакція RESTRICT: клієнта K-310, у якого є видачі, видалити не можна.
BEGIN;
DELETE FROM customers WHERE customer_number = 'K-310';
-- Очікувано: update or delete on table "customers" violates RESTRICT setting
-- of foreign key constraint "rentals_customer_number_fkey" on table "rentals"
ROLLBACK;
```

Результат: файл `data-changes.sql` і розділ 6 звіту - таблиця «зміна - рядків у SELECT - повідомлення СУБД - завершення» і таблиця «зв'язок - реакція - що сталося з пов'язаними рядками».

Крок 7. Явна транзакція: COMMIT, ROLLBACK і помилка

- Обрати операцію власної області, яка змінює щонайменше два рядки в різних таблицях і має сенс лише повністю: повернення велосипеда, оплата замовлення, переведення працівника. Створити файл `lab-02/transactions.sql` з трьома частинами. Команди виконують по одній, щоб після кожної бачити повідомлення СУБД.
- Частина 1, COMMIT: BEGIN; зміни операції; COMMIT; SELECT показує обидві зміни.
- Частина 2, ROLLBACK: та сама операція для іншого рядка; BEGIN; зміни; SELECT - у цьому сеансі зміни видно; ROLLBACK; SELECT - стан такий, як перед BEGIN.
- Частина 3, помилка всередині транзакції: BEGIN; перша зміна виконується; друга порушує обмеження схеми; наступний SELECT відхиляється повідомленням `current transaction is aborted, commands ignored until end of transaction block`; на COMMIT СУБД відповідає `ROLLBACK`; SELECT після цього показує, що першої зміни теж немає.

```
-- Частина 3. Повернення велосипеда B-31 з помилкою в назві пункту.
BEGIN;
UPDATE rentals SET returned_at = '2026-04-01 11:00'
WHERE inventory_number = 'B-31' AND rented_at = '2026-03-31 18:30';     -- UPDATE 1
UPDATE bikes SET station_name = 'Пункт №5'
WHERE inventory_number = 'B-31';
-- Очікувано: insert or update on table "bikes" violates foreign key constraint
"bikes_station_name_fkey"
SELECT returned_at FROM rentals
WHERE inventory_number = 'B-31' AND rented_at = '2026-03-31 18:30';
-- Очікувано: current transaction is aborted, commands ignored until end of transaction
block
COMMIT;                                                                  -- відповідь СУБД:
ROLLBACK
SELECT returned_at FROM rentals
WHERE inventory_number = 'B-31' AND rented_at = '2026-03-31 18:30';     -- NULL:
повернення не записано
```

Результат: файл `transactions.sql` і розділ 7 звіту - для кожної частини команди, повідомлення СУБД і стан даних після завершення транзакції.

Крок 8. Втрачене оновлення у двох сеансах

- Обрати числовий стовпець власної області, який двоє користувачів можуть змінювати, спираючись на прочитане значення: залишок, баланс, ціну, кількість місць. Записати початкове значення і дві зміни, наприклад «мінус 10 %» і «плюс 30».
- Відкрити два вікна Query Tool до бази `dbsa`: сеанс А і сеанс Б. Команди виконують по одній, строго в порядку кроків часової шкали, перемикаючись між вікнами. Команди обох сеансів записати у файл `lab-02/lost-update.sql` з коментарями `-- Крок N. Сеанс А` чи `-- Крок N. Сеанс Б`.
- Перед кожним дослідом значення повертають до початкового однією командою UPDATE.
- Дослід 1, READ COMMITTED. Кожен сеанс у своїй транзакції читає значення, рахує нове сам і записує готове число.

```
-- Дослід 1. READ COMMITTED. Ціна тарифу Д1 перед дослідом - 250.
-- Менеджер А знижує ціну на 10 %, менеджер Б підвищує на 30 грн. Має бути 255.

-- Крок 1. Сеанс А
BEGIN;
SELECT price FROM tariffs WHERE tariff_code = 'Д1';     -- 250.00

-- Крок 2. Сеанс Б
BEGIN;
SELECT price FROM tariffs WHERE tariff_code = 'Д1';     -- 250.00

-- Крок 3. Сеанс А: 250 - 10 % = 225
UPDATE tariffs SET price = 225 WHERE tariff_code = 'Д1';

-- Крок 4. Сеанс Б: 250 + 30 = 280; команда чекає
UPDATE tariffs SET price = 280 WHERE tariff_code = 'Д1';

-- Крок 5. Сеанс А
COMMIT;                                                 -- у вікні сеансу Б з’являється
UPDATE 1

-- Крок 6. Сеанс Б
COMMIT;
SELECT price FROM tariffs WHERE tariff_code = 'Д1';     -- 280.00 замість 255.00
```

Крок	Сеанс А (-10 %)	Сеанс Б (+30 грн)
1	BEGIN; ціна 250	
2		BEGIN; ціна 250
3	записує 225	
4		записує 280 і чекає
5	COMMIT	запис виконано
6		COMMIT; у базі 280

- Дослід 2, REPEATABLE READ. Ті самі кроки, але обидва сеанси відкривають транзакцію командою `BEGIN ISOLATION LEVEL REPEATABLE READ;`. Після COMMIT сеансу А команда сеансу Б завершується помилкою `could not serialize access due to concurrent update`. Сеанс Б виконує ROLLBACK і повторює транзакцію з початку: читає нове значення (225), рахує, записує 255, COMMIT.
- Дослід 3, зміна відносно поточного значення. На READ COMMITTED сеанс А виконує `UPDATE tariffs SET price = price * 0.9 WHERE tariff_code = 'Д1';`, сеанс Б - `UPDATE tariffs SET price = price + 30 WHERE tariff_code = 'Д1';`. Сеанс Б так само чекає, але після COMMIT сеансу А рахує від уже зміненого значення: 255 без помилки.

- Для кожного досліду заповнити часову шкалу: крок, команда і результат у кожному сеансі, позначка кроку, де сеанс чекав на блокування, значення в базі наприкінці. Записати, яким значення мало бути і яким воно стало.

Результат: файл `lost-update.sql` і розділ 8 звіту - три часові шкали і висновок: чому в досліді 1 зміну втрачено і чим два інші способи її зберігають.

Крок 9. Повний прогін у порожній базі

- У pgAdmin створити порожню базу `dbsa_check` (Databases - Create - Database) і відкрити для неї Query Tool.
- Виконати всі файли `*.up.sql` з каталогів `lab-01/migrations` і `lab-02/migrations` за зростанням номерів.
- Якщо в роботі є міграція: виконати її файл down і скрипт перевірки схеми (обмеження, яке вона додає, зникло), потім знову файл up і скрипт перевірки схеми (обмеження на місці).
- Виконати `seed-data.sql`, потім скрипт підрахунку рядків: результат збігається з розділом 2 звіту.
- Виконати `seed-data.sql` удруге і знову скрипт підрахунку рядків: результат той самий.
- Виконати кожен запит із `queries.sql` окремо: кількість рядків збігається з коментарями у файлі і з розділами 4 і 5 звіту.
- Записати в розділ 9 звіту, чим завершився кожен пункт. Розбіжності означають, що дані в базі `dbsa` відрізняються від скрипта наповнення; їх виправляють у скрипті або в запиті і повторюють прогін. Після перевірки базу `dbsa_check` можна видалити.

Результат: розділ 9 звіту.

Структура каталогу і здача через Git

```
<репозиторій студента>/
├── README.md                                  опис предметної області (лекція 1)
├── compose.yaml                               файл опису середовища, спільний для робіт
├── lab-01/                                    робота №1
└── lab-02/
    ├── README.md                              звіт за шаблоном
    ├── seed-data.sql                          скрипт наповнення
    ├── queries.sql                            запити кроків 4 і 5
    ├── data-changes.sql                       зміна і видалення даних, реакції на видалення
    ├── transactions.sql                       явні транзакції
    ├── lost-update.sql                        команди двох сеансів
    └── migrations/                            лише якщо на кроці 1 схему змінено
        ├── 002_add_station_name_unique.up.sql
        └── 002_add_station_name_unique.down.sql
```

- Звіт - файл `lab-02/README.md`, створений зі шаблону звіту (`lab-02/report-template.md`).

- Файли міграції роботи №1 не редагують. Зміна схеми в цій роботі - нова пара файлів у `lab-02/migrations` з наступним номером; назви файлів - за правилами роботи №1: тризначний номер, підкреслення, короткий опис англійською малими літерами, суфікс `.up.sql` або `.down.sql`.
- Скрипт наповнення не є міграцією: схему він не змінює і в каталог `migrations` не потрапляє.
- Усі SQL-файли і звіт зберігаються в кодуванні UTF-8.
- Робота виконується кількома змістовними комітами: щонайменше окремі коміти для міграції (якщо вона є), скрипта наповнення з перевіркою ідемпотентності, запитів, змін даних і транзакцій, дослідів із двома сеансами. Повідомлення коміту описує зміну: «Додано скрипт наповнення і перевірку повторного запуску», а не «update».
- Історія комітів є частиною оцінювання: вона підтверджує самостійність роботи і те, що закомічені файли міграції не змінювалися.
- На перевірку подається стан репозиторію в основній гілці. Терміни здачі і спосіб повідомлення про готовність роботи оголошує викладач.

Зміст звіту

Звіт `lab-02/README.md` містить розділи в такому порядку (усі вони вже є в шаблоні звіту (`lab-02/report-template.md`)):

1. Ключі для повторного завантаження - таблиця «таблиця - обмеження - стовпці ON CONFLICT»; посилання на файли міграції, якщо схему змінено, і результат циклу up - down - up.
2. Скрипт наповнення - посилання на файл, кількість рядків у кожній таблиці після першого запуску, рядки, додані для вимог до даних.
3. Ідемпотентність - таблиця «дія - повідомлення СУБД - кількість рядків у таблицях», обмеження, яке захистило кожну таблицю.
4. Запити з фільтрацією і сортуванням - таблиця «питання - використані конструкції - кількість рядків»; пара запитів із NULL і пояснення трозначною логікою.
5. Запити зі з’єднанням таблиць - три запити з кількістю рядків і поясненням; прочитання з’єднання 2 в логічному порядку; порівняння з «поганою» таблицею роботи №1.
6. Зміна і видалення даних - таблиця змін і таблиця реакцій на видалення.
7. Явні транзакції - COMMIT, ROLLBACK і помилка всередині транзакції: команди, повідомлення СУБД, стан даних.
8. Втрачене оновлення - три часові шкали і висновок.
9. Повний прогін у порожній базі - результати кроку 9.

Критерії приймання

- У каталозі `lab-02` є звіт з усіма розділами і файли `seed-data.sql`, `queries.sql`, `data-changes.sql`, `transactions.sql`, `lost-update.sql`; якщо схему змінено - пара файлів

міграції з наступним номером.

- Для кожної таблиці названо обмеження, на якому тримається повторне завантаження; таблиці із сурогатним ключем мають UNIQUE на потенційному ключі.
- Міграція, якщо вона є, проходить цикл up - down - up; файли міграції роботи №1 не змінено.
- Скрипт наповнення містить для кожної таблиці INSERT з переліком стовпців і ON CONFLICT із переліком стовпців обмеження; інших команд зміни даних і схеми в ньому немає; дані відповідають вимогам кроку 2.
- Повторний запуск скрипта наповнення завершується без помилки, а кількість рядків у кожній таблиці до і після збігається; повідомлення `INSERT 0 0` і помилку команди без ON CONFLICT записано у звіті.
- Шість запитів на питання замовника разом використовують порівняння з AND або OR, BETWEEN, IN, LIKE, IS NULL або IS NOT NULL, ORDER BY з LIMIT; змішані AND і OR записано з дужками; кожна умова і лишає, і відкидає хоча б один рядок; періоди для дати й часу записано напівінтервалом; LIMIT не лишає нічиїх.
- Пара запитів із NULL показує загублені рядки і виправлення; різницю пояснено трозначною логікою.
- Три з'єднання виконано за парами «зовнішній ключ - первинний ключ» із псевдонімами таблиць; для кожного пояснено кількість рядків; ліве з'єднання знаходить рядки без пари; з'єднання 2 прочитано в логічному порядку і порівняно з «поганою» таблицею.
- Кожна зміна кроку 6 виконана в транзакції з попереднім SELECT, і кількість рядків SELECT збігається з повідомленням `UPDATE N` чи `DELETE N`; кожну реакцію на видалення зі схеми показано на даних і скасовано ROLLBACK.
- На кроці 7 показано підтвердження, скасування і помилку всередині транзакції; для помилки записано повідомлення `current transaction is aborted` і те, що перша зміна не збереглася.
- На кроці 8 є три досліди з часовими шкалами: втрачене оновлення на READ COMMITTED, помилка серіалізації і повтор транзакції на REPEATABLE READ, правильний результат зміни відносно поточного значення.
- Повний прогін у порожній базі `dbsa_check` проходить без помилок: кількість рядків після двох запусків скрипта наповнення однакова і збігається зі звітом, кількість рядків запитів збігається з коментарями.
- Назви таблиць, стовпців, псевдонімів і файлів міграції - англійською, малими літерами, через підкреслення, без лапок і транслітерації.
- Історія Git складається з кількох змістовних комітів.
- На захисті студент відповідає на контрольні питання на прикладі власної бази.

Контрольні питання до захисту

1. Яке обмеження захищає від дублікатів названу викладачем таблицю під час повторного запуску скрипта наповнення? Що станеться, якщо цього обмеження не буде?
2. Чому стовпець ідентичності як первинний ключ не захищає від повторного завантаження і що для цього потрібно?

3. Після змін кроку 6 скрипт наповнення запустили ще раз. Що станеться з видаленим рядком і зі зміненими значеннями? А якби в скрипті було ON CONFLICT DO UPDATE?
4. Покажіть у своєму запиті умову, яка губить рядки з NULL. Чому ці рядки не потрапили в результат і як їх отримати?
5. Чому межу періоду для стовпця з датою й часом записують напівінтервалом? Чи потрібен напівінтервал для стовпця типу `date`?
6. Скільки рядків повертає ваше внутрішнє з'єднання і які правила схеми гарантують саме таку кількість? Що зміниться, якщо зовнішній ключ допускає NULL?
7. Прочитайте в логічному порядку виконання названий викладачем запит зі свого файлу і сформулюйте питання, на яке він відповідає.
8. Ліве з'єднання велосипедів і видач з умовою `WHERE r.tariff_code = 'Д1'` повертає 3 рядки, а з тією самою умовою в ON - 6. Чому?
9. Навіщо перед UPDATE і DELETE виконувати SELECT з тією самою умовою і навіщо робити це всередині транзакції?
10. Що сталося з першою зміною частини 3 кроку 7 і чому COMMIT завершився скасуванням?
11. Чому в досліді 1 сеанс Б чекав на кроці 4 і чому після очікування зміна сеансу А все одно загубилася?
12. Назвіть два способи не втратити зміну і ціну кожного.
13. Правило «у клієнта не більше однієї незавершеної видачі» перевіряють у транзакції: читають незавершені видачі клієнта і лише потім додають нову. Чому на REPEATABLE READ два пункти одночасно можуть видати клієнтові по велосипеду і що змінює SERIALIZABLE?
14. Два сеанси змінюють ті самі два рядки в протилежному порядку. Що зробить PostgreSQL і як цього уникнути?

Типові помилки та як їх уникнути

- Після перезавантаження комп'ютера pgAdmin не підключається до бази. Протидія: запустити Docker Desktop і виконати в корені репозиторію `docker compose up -d`; інші проблеми середовища - за розділом «Типові помилки та як їх уникнути» роботи №1.
- Скрипт наповнення завершується помилкою `insert or update on table ... violates foreign key constraint`. Протидія: команди INSERT мають іти в порядку створення таблиць; якщо порядок правильний, значення ключа не збігається з рядком батьківської таблиці - пробіл, інша літера або латинська літера замість кириличної (`B-12` і `B-12` - різні значення).
- Команда з ON CONFLICT завершується помилкою `there is no unique or exclusion constraint matching the ON CONFLICT specification`. Протидія: у переліку ON CONFLICT - рівно стовпці первинного ключа чи UNIQUE; якщо такого обмеження немає, його додають міграцією на кроці 1.
- Повторний запуск скрипта додав рядки. Протидія: у таблиці із сурогатним ключем немає UNIQUE на потенційному ключі або в команді записано ON CONFLICT без переліку стовпців; обмеження додають міграцією, а стовпці вказують явно.

- Після зміни скрипта наповнення дочірні рядки посилаються не на ті батьківські рядки або СУБД відхиляє їх через зовнішній ключ. Протидія: номери сурогатних ключів у скрипті відповідають першому запуску в порожній базі; базу `dbsa` очищають циклом down - up і наповнюють знову, а збіг перевіряють на кроці 9.
- Міграція з UNIQUE завершується помилкою `could not create unique index ...` з уточненням `is duplicated`. Протидія: у таблиці вже є однакові значення; міграцію виконують до наповнення, а в заповненій базі спершу очищають таблиці циклом down - up.
- Запит з умовою `= NULL` не повертає жодного рядка. Протидія: рядки з NULL шукають лише через IS NULL та IS NOT NULL.
- BETWEEN для стовпця з датою й часом губить події останнього дня періоду. Протидія: напівінтервал `>=` початок і `<` початок наступного періоду.
- Змішані AND і OR без дужок повертають зайві рядки. Протидія: AND виконується раніше за OR, тому умову з OR беруть у дужки або замінюють рівності через OR на IN.
- Запит завершується помилкою `column "Trek" does not exist`. Протидія: текст і дати записують в одинарних лапках; подвійні лапки означають назву стовпця.
- LIKE не знаходить рядків. Протидія: у PostgreSQL LIKE розрізняє великі й малі літери; без `%` шаблон працює як рівність.
- LIMIT повертає різні рядки від запуску до запуску. Протидія: LIMIT пишуть лише з ORDER BY; до стовпця, що може повторюватися, додають стовпці ключа.
- Сортування за спаданням стовпця з NULL ставить першими рядки без значення. Протидія: NULLS LAST.
- Запит зі з'єднанням завершується помилкою `column reference ... is ambiguous`. Протидія: кожен стовпець записують із псевдонімом таблиці.
- З'єднання повертає більше рядків, ніж є в таблиці із зовнішнім ключем. Приклад: три видачі велосипеда B-40, з'єднані з велосипедами умовою `ON b.station_name = r.station_name`, дають 6 рядків - кожну видачу поставлено поруч з усіма велосипедами пункту. Протидія: умова ON - пара «зовнішній ключ - первинний ключ» (`b.inventory_number = r.inventory_number`), для складеного ключа - з усіма його стовпцями; кількість рядків звіряють із таблицею із зовнішнім ключем.
- Ліве з'єднання не показує рядків без пари. Протидія: умову для правої таблиці записують в ON, а не у WHERE.
- UPDATE чи DELETE без WHERE або з помилковою умовою змінив зайві рядки. Протидія: зміни виконують за шаблоном кроку 6 - BEGIN, SELECT, зміна, SELECT, COMMIT або ROLLBACK. Якщо зміну вже підтверджено, базу очищають циклом down - up і наповнюють знову.
- Кожна команда у вікні Query Tool відхиляється повідомленням `current transaction is aborted`. Протидія: у транзакції сталася помилка; виконати ROLLBACK і повторити транзакцію.
- На кроці 7 немає повідомлення `current transaction is aborted`, а ROLLBACK повідомляє `there is no transaction in progress`. Протидія: у меню Execute options позначено Auto rollback on error?, і pgAdmin сам скасував транзакцію; зняти позначку і повторити частину 3.

- Вікно Query Tool «зависло» на команді UPDATE. Протидія: команда чекає на блокування рядка незавершеною транзакцією в іншому вікні; виконати там COMMIT або ROLLBACK. Перезапускати pgAdmin не потрібно.
- Втрачене оновлення не відтворюється. Протидія: сеанс Б має прочитати значення до COMMIT сеансу А, а обидва сеанси в досліді 1 записують готове число, а не `price + 30`; кроки виконують строго за часовою шкалою.
- У досліді 2 немає помилки серіалізації. Протидія: транзакцію сеансу Б відкрито без `ISOLATION LEVEL REPEATABLE READ` або її перше читання відбулося вже після COMMIT сеансу А.
- Кількість рядків запитів у звіті не збігається з повторним виконанням у базі `dbsa`. Протидія: після кроку 6 дані змінено; кількість рядків фіксують одразу після наповнення, а перевіряють у порожній базі на кроці 9.

Забезпечення роботи

- Скрипт підрахунку рядків (`lab-02/count-rows.sql`) - виконується в pgAdmin без змін.
- Шаблон звіту (`lab-02/report-template.md`) - копіюється як `lab-02/README.md`.
- З роботи №1: файл опису середовища (`lab-01/compose.yaml`) і скрипт перевірки схеми (`lab-01/check-schema.sql`).
- Програмне забезпечення: Docker Desktop (Windows, macOS) або Docker Engine із Docker Compose (Linux), у Windows - WSL 2; образ `postgres:18.6`; pgAdmin 4 версії 9.17 з параметрами Query Tool за замовчуванням (Auto commit? - позначено, Auto rollback on error? - не позначено); Git.
- Зразки команд і запитів - колоди слайдів лекції 5 і лекції 6.
- Роздатковий матеріал лекцій: бланк часової шкали двох сеансів (лекція 5); шпаргалка «шаблон запиту і логічний порядок виконання» і таблиця «істина - хибність - невідомо» для WHERE і CHECK (лекція 6).

Рекомендовані джерела

- Beaulieu A., розділи про наповнення і зміну даних, основи запитів, фільтрацію, запити до кількох таблиць і транзакції.
- Silberschatz A. та ін., розділи 3-4 (Introduction to SQL, Intermediate SQL) і розділ про транзакції та рівні ізоляції.
- Forta B., уроки про вибірку, сортування, фільтрацію, з’єднання таблиць, додавання, зміну і видалення даних - для самостійного надолуження синтаксису.
- Берко А. Ю. та ін. Кн. 2, розділи про мову SQL і транзакції.
- Офіційна документація PostgreSQL, розділи Data Manipulation, Queries і Concurrency Control, а також опис команди INSERT (підрозділ ON CONFLICT Clause).